Feasibility Analysis and Requirements Specification

Smart University Website

Group S | Wai Yan Lynn (67111411) · Pyae Sone (67111336) · Nang Ying Noom Lao (67111378) · Myat Hsu Hnin (68111423)
IT Project I and II (BC362916 + BC362914) | Instructor: Dr. Aftab Alam Janisar | Week 05 | 2 October 2026
The target institution is a fictional bilingual university, referred to as the University. No real institutional data is used.

Section 1: Problem Summary and Target Domain

1.1 Problem Restatement

A university website is built for the university to publish information, not for the student to get things done. This causes three problems.

- **The website accepts no action.** A student can read how to request a transcript, but cannot send the request, check if it is ready, or book the advisor meeting that the page tells them to book. The same is true for every campus space a student can book, such as a study room, an instrument room, a club activity room or a seat at a workshop. The website stops right where the student's real task starts, so the task turns into an email, a phone call or a queue at a counter.
- **The rules are published but hard to search.** Rules and fee information sit inside PDF handbooks and old announcement posts, and the site search only matches page titles, not the rule inside the file. A student has to know which document holds the answer before they can find it, so most students ask a senior instead, and unofficial answers spread faster than official ones.
- **Help stops when a staff member is away.** The office keeps no shared record of a case, so what a student explained is only in the memory of the staff member who helped them. If that person is away the next day, a student who is collecting visa documents has to explain everything again, and the next staff member repeats the same search and sometimes gives a different answer.

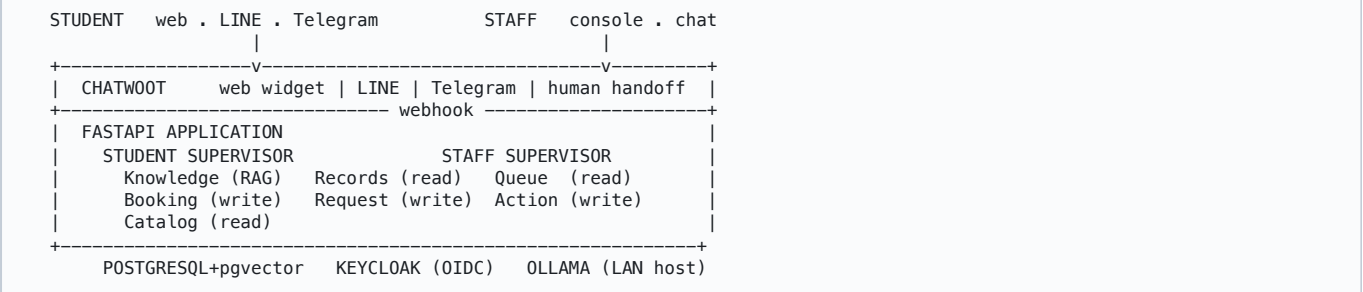
1.2 Project Objectives

1. Index the regulation corpus and answer at least 80 percent of a 50-question test set correctly, with a source citation on every answer.
2. Move the student's common tasks inside the website: ask a policy question, read their own record, book any campus resource, and submit and track any service request.
3. Give every booking and every request a durable state and a visible history, so that no student travels to campus to ask about progress.
4. Preserve conversation context across staff changes and across channels, so that a student never restates a case the institution already holds.
5. Give staff the same conversational interface over their own queues, and keep every component free and open source with the model running on team hardware.

1.3 Target User Profile and Operational Environment

The environment is the student services function of the University: an office of three to six staff, advisors holding weekly office hours, a facilities team owning bookable rooms and equipment, a library, and a student body that already talks to each other through chat apps. The office is open on weekdays only, and the number of questions rises sharply around registration, fee deadlines and result days. **Students** are comfortable with chat apps and not with university software. They use a phone, often outside office hours, in English or Thai, and they will not read a manual. **Staff** are comfortable with office software and a shared inbox and are not developers. **Advisors and facility owners** want to spend no time administering a calendar. **The administrator** is technical and owns accounts, roles and the deployment.

1.4 System Context



Retrieval-augmented generation (RAG) means the model answers only from text the system retrieves first. OpenID Connect (OIDC) is the protocol Keycloak uses to issue and verify login tokens. Ollama is an open-source runtime that serves an open-weight language model over HTTP.

Section 2: Multi-Dimensional Feasibility Analysis

This section uses the four dimensions named in the assignment brief. The writing guide's six dimensions map into them: Resource Feasibility sits in 2.2, and Hardware Access and Risk Identification and Mitigation sit in 2.4.

2.1 Technical Feasibility

Stack, all free or open source. Python 3.12 with FastAPI and Uvicorn for the backend, because the agent ecosystem is strongest in Python and every agent turn waits on a model call. LangGraph with langgraph-supervisor for the agents, giving an explicit state graph and two supervisors in one process with different tool permissions. Ollama serving an open-weight model over the local network, so there is no API key and no quota. PostgreSQL 16 with pgvector, so one engine holds both transactional records and document vectors. Keycloak for identity, which removes all credential handling code. Self-hosted Chatwoot, which supplies the web widget, the LINE and Telegram connectors and human takeover in one service. React with Vite as a single application with Keycloak role-gated routes. Docker Compose with an ngrok tunnel for channel webhooks. pytest and Locust for testing.

Architecture realism. The system is one application process, one database and three supporting services. There is no message broker, no microservices and no Kubernetes. The multi-agent part is two routing graphs inside one process, so when an agent fails we debug one program instead of many. We keep the long feature list affordable by making features share code. Every bookable thing is one resource row with a slot table, and every trackable thing is one request row with a type and a state. Two pieces of code therefore cover eight features, and we write each one only once.

API access and rate limits. The prototype has no external paid dependency and no rate limit, because the model runs on team hardware. One of our laptops runs Ollama on the local network and the other three point their settings at it. *Production note:* running the model on our own laptops is fine for a prototype with no budget, but it is not enough for a real service. A real deployment would need a graphics card or a paid provider such as OpenRouter. All model calls go through one adapter file, so changing provider later means changing settings and that one file, not the agents.

Skills and gaps. Some of us have used parts of this stack before, in coursework and in an internship, so we are not starting from zero on FastAPI, PostgreSQL, LangGraph, RAG, Keycloak and Chatwoot. Whoever already knows a tool will share it with the rest of the team while we build, and the first two weeks leave time for that. Three real gaps remain. **Tool calling with a local model** is the biggest one. Small local models follow tool instructions less reliably than the hosted ones, so in Week 2 we test two or three models and pick one from the results, before we write any agent. **Measuring answer quality** is new to us, so we write the test set before the agents and have two people score the answers separately. **Thai search and load testing** are also new. We use a multilingual embedding model with Thai questions in the test set from Week 2, and Locust for load testing, which takes about one day to learn.

2.2 Operational and Resource Feasibility

Students already answer questions through chat on a phone, so students do not have to learn a new habit, and LINE is a channel this group opens daily. The adoption risk is with the staff, not the students. An assistant that answers questions takes away work they own today, and a console is a new tool in a workflow that runs on a shared inbox. Our design answers both. The assistant decides nothing, and it passes the conversation to a person when the student asks or when it is not sure. The staff-side agent also lets a staff member work through chat instead of learning a console. Staff training is one page covering three actions, which are take over a conversation, change a request state, and release the conversation. Students need no training.

Tooling, hosting and budget

Total monetary budget: zero. Docker images for all services, Ollama on team hardware for inference, ngrok free tier for channel webhooks, Tailscale free tier if the campus network blocks machine-to-machine traffic, and the GitHub free plan. The real limit is memory, not money. We own two laptops with 16 GB and two with 8 GB, so one 16 GB laptop runs Ollama only, the other runs the full container stack and is the demo machine, and the two 8 GB laptops run only the application and PostgreSQL and share the heavier services. Nobody runs the model and the full stack on the same laptop, because that is the setup that runs out of 16 GB.

Team allocation, split by layer rather than by feature. Track 1 owns infrastructure and identity and publishes the database schema and token claims. Track 2 owns the English and Thai corpus, indexing, the test set and all measurement, and publishes the retrieval function signature. Track 3 owns both supervisors, all seven agents and every tool, and publishes the chat endpoint. Track 4 owns the React application, the public pages, the portal, the staff console and the three channels. The team agrees who takes which track in Week 1. If four people edit the same agent file we get merge

conflicts, and if each person works behind an agreed interface we do not. So we agree the four interfaces in W1 and each track builds against a simple fake version of the others until the real one is ready.

2.3 Time Feasibility

Phase	Delivered	Done means
W1–W2	Compose stack, schema, Keycloak realm, Ollama host, interfaces agreed. Test set written and models compared before any agent code.	A user signs in, and the model is chosen on measured evidence.
W3–W6	Corpus indexed. Knowledge, Records, Booking and Request agents with supervisor routing, slot locking, the four-state request lifecycle and notifications. Portal views.	The core journey works end to end, and the concurrent booking test shows zero double-bookings.
W7–W9	Catalog agent, Chatwoot with Telegram then LINE, staff supervisor and console, access-control negative suite, load test, final scoring, usability test.	One conversation runs across three channels, and every target in Section 4.3 is measured.
W10	**Buffer. No new features.** Defect fixing, documentation, demonstration rehearsal.	We have room to fall behind without cutting the main features.

Two of the ten weeks add no new features. W2 is for measuring, and W10 is for fixing bugs. If we fall behind, we have already decided what to drop first: LINE first, then the staff console interface, then the Catalog agent, then the staff-side supervisor. **Deferred to Project II:** cloud deployment with a paid inference provider, course add and drop with prerequisite and timetable-clash checking, payment processing, integration with a real student information system, and full-text retrieval inside library holdings.

2.4 Data Feasibility and Key Risks

We use no real university data, and we do not need any. We have no partner university, and asking for real student records would raise a privacy problem we cannot handle as students. Every data set is constructed: the regulation corpus is authored from publicly published university handbooks in aligned English and Thai versions; student records, bookable resources and catalog entries are generated with synthetic names and identifiers; and the evaluation sets are 50 graded test questions and 100 labelled routing messages, frozen before agent development. All of it is committed to the repository, so every evaluation run is reproducible.

Privacy and security. We build the system as if the data were real, because getting access control right is the part that matters most. Every agent tool takes the subject identifier from the verified login token and never from the message text, the staff tool set is unreachable from a student session, every staff write is capped and recorded, personal identifiers are excluded from logs, and all browser traffic uses TLS. **Hardware:** four team laptops as described above, with no graphics processor, which is why we only test models that run on normal laptop memory.

Technical and schedule risks, with mitigation

ID	Risk	L / I	Mitigation
R1	The local model is unreliable at tool calling, so agents pick the wrong tool or emit malformed arguments.	High / High	In Week 2 we test tool calling on two or three models before we write any agent. We keep the tool definitions small and simple. A checking layer rejects bad arguments and tries once more. If no local model is good enough, we switch the adapter to a paid provider and say so in the report.
R2	Prompt injection: text in a document or a message instructs the agent to act or to reveal another user's data.	Medium / High	Tools take the subject from the token, never from text. Each agent has an explicit tool allowlist and no tool accepts free-form queries. A negative suite attempts cross-account access on every tool in continuous integration.
R3	Scope creep, because a university offers an endless supply of plausible additions.	High / High	Section 6 freezes the list. Adding something means removing something of the same size, and we only accept it if it reuses the booking code or the request code.
R4	Integration failure between four parallel tracks, or a campus network that blocks access to the model host.	Medium / High	We agree four interfaces in W1 and each track builds against a simple fake version of the others. We test network access in W2, not on demo day, and our backups are a free mesh network service and then a shared phone hotspot.
R5	Schedule slip or uneven contribution across a four-person team.	Medium / High	Each track carries a weekly milestone with a binary done condition, and the four tracks demonstrate against the frozen interfaces at a weekly integration checkpoint. One feature works end to end by W4, W10 adds no new features, and we agree in advance what to drop first instead of deciding under pressure.

R6	Thai quality falls far below English, or the agent gives a confident answer the documents do not support.	Medium / Medium	Thai questions are in the test set from W2, so we see the gap early. If Thai answers are weak, the agent shows the Thai passage it found with its source instead of writing a summary. Every answer carries a source, and if nothing scores above the similarity threshold the agent refuses instead of making something up.

Section 3: Feasibility Recommendation

PROCEED WITH ADJUSTMENTS

The project is feasible for a four-person team within Project I, with the adjustments below. The system is one application process, one database and three supporting services, and we have used most of these tools before, so the main risk is the quality of the local model rather than the infrastructure. Every tool is free or open source and the model runs on our own laptops, so the budget is zero. The feature list looks wide, but eight features share the same two pieces of code, one for booking a slot and one for tracking a request, and we write each one only once. Three changes keep the schedule honest: we choose the model by testing it in Week 2 instead of assuming, we support English and Thai only and measure Thai from the start, and every feature that would need new code of its own, including payments, timetable-clash checking and full-text library search, moves to Project II. With those adjustments the project is ready to move to requirements and system architecture.

Section 4: Stakeholders and Requirements Specification

4.1 Stakeholder Register

Class	Stakeholder	Interest and influence
Primary users	Enrolled students; student services staff	Students want an immediate correct answer and a task finished without visiting a counter. Staff want fewer repeat questions and any case with its full history. Both have high influence, because the system has no value if either group does not use it.
Secondary users	Advisors; facility and library staff; head of student services	Advisors and facility owners want one booking per published slot and own the resource data. The head of office wants volume and turnaround visibility and sponsors the deployment decision. Medium influence.
Administrators	IT administrator	Manages accounts and roles, runs the container stack and the model host, keeps the corpus current. High influence, because they control deployment, access and account provisioning.
External actors	Ollama model host; LINE Messaging API; Telegram Bot API; prospective students	The model host generates every reply, so it is high influence and a single point of failure. The channel APIs are low influence, because the web widget works without them.

4.2 Functional Requirements

A *bookable resource* is an advisor office hour, study room, instrument room, club activity room or workshop seat. A *service request* is a document request, library hold or maintenance report. One requirement therefore covers every instance of its type, which is how the code is written. The test method is stated in brackets after each requirement.

1. **FR-01** The system shall authenticate every user through Keycloak using OpenID Connect and shall reject with HTTP 401 any request to a protected endpoint carrying no valid access token. *(Tested with an absent, an expired and a valid token.)*
2. **FR-02** The system shall assign each account exactly one of the roles student, staff, advisor or administrator and shall return HTTP 403 when an account calls an endpoint its role does not permit. *(Every endpoint called once per role.)*
3. **FR-03** The system shall route every student message to exactly one specialist agent among Knowledge, Records, Booking, Request and Catalog, and shall store the chosen route with the message identifier. *(Routing log read after the 100-message labelled set runs.)*
4. **FR-04** The system shall answer a policy question only from passages retrieved from the indexed corpus and shall display the source document title and section with every answer. *(All 50 test answers inspected. Zero without a citation.)*
5. **FR-05** The system shall refuse to answer and shall offer a human handoff when no retrieved passage scores above the configured similarity threshold. *(10 questions with no corpus coverage. All 10 refuse.)*
6. **FR-06** The system shall accept student messages in English and in Thai and shall reply in the language of the incoming message. *(20 English and 20 Thai questions. Language matches in all 40.)*

7. **FR-07** The system shall return the signed-in student's own timetable, registration status and outstanding fee balance, and shall refuse any request that names another student. *(30 other-student retrieval attempts. 30 refusals expected.)*
8. **FR-08** The system shall list the available slots of any bookable resource for a given date range. *(Compared against the database for 10 ranges across 3 resource types.)*
9. **FR-09** The system shall reserve a selected slot inside a single database transaction and shall reject any second reservation of a slot already taken. *(200-request concurrent test on one slot. Exactly one success.)*
10. **FR-10** The system shall allow a student to cancel or reschedule only their own reservation, and only before the cut-off configured for that resource type. *(Tested before the cut-off, after it, and on another student's reservation.)*
11. **FR-11** The system shall accept a service request of a configured type, shall hold it in exactly one of the states submitted, under review, ready or closed, and shall record a timestamped history entry for every state change. *(One request of each type moved through all four states, history read back.)*
12. **FR-12** The system shall notify the requesting student on their originating channel within 60 seconds of a change to the state of their reservation or request. *(Delivery measured on web, LINE and Telegram.)*
13. **FR-13** The system shall hand a conversation to a human staff member on student request or on low routing confidence and shall generate no further automatic reply until the staff member releases it. *(Handoff triggered, silence confirmed for 10 further messages.)*
14. **FR-14** The system shall exchange messages on the web widget, LINE and Telegram through Chatwoot and shall present one continuous conversation history per student across all three channels. *(Conversation started on web, continued on LINE and Telegram, one history read back.)*
15. **FR-15** The system shall provide a staff-side agent that lists a staff member's open queues and performs state changes and cancellations on request, shall require explicit confirmation before any action affecting more than one record, and shall write an audit record of actor, action, target and timestamp for every agent write. *(One single and one bulk action performed as staff, prompt confirmed, both audit rows read.)*

4.3 Non-Functional Requirements

Performance targets are set for a locally hosted open-weight model on consumer hardware, which is slower than a hosted API. The system streams its reply, so first visible token and completion time are both stated.

1. **NFR-01 Performance.** 95th-percentile time to first visible token shall be 4 seconds or less, and to complete reply 15 seconds or less for a retrieval answer and 8 seconds or less for a records lookup, over 100 requests. The system shall sustain 10 concurrent conversations with no failed request.
2. **NFR-02 Security.** Authentication shall use OpenID Connect with RS256-signed tokens expiring within 15 minutes. All browser traffic shall use TLS 1.2 or higher. The application shall store no password. Of 30 cross-account access attempts and 10 attempts to reach a staff tool from a student session, 0 shall succeed.
3. **NFR-03 Usability.** A first-time student shall complete "book a study room" in 90 seconds or less and in 4 messages or fewer, with no written instructions. Of 5 test users, at least 4 shall succeed.
4. **NFR-04 Availability and reliability.** The stack shall start from one `docker compose up` within 5 minutes on a clean machine. A container restart shall lose no reservation and no submitted request. If the model host is unreachable, the system shall return a service message within 10 seconds rather than waiting indefinitely.
5. **NFR-05 Accuracy.** Routing accuracy shall be 90 percent or higher on a separate test set of 100 labelled messages. At least 80 percent of the 50 test questions shall be answered correctly with a correct citation, scored by two independent raters and reported separately for English and Thai.

Section 5: User Stories and Use Cases

5.1 User Stories

1. As a **student**, I want to ask about a regulation in chat and receive the answer with the rule it came from, so that I do not have to search PDF handbooks or rely on what a senior told me.
2. As a **student**, I want to book an advisor meeting, a study room or an instrument room inside the same conversation, so that I do not queue for a slot someone else already took.
3. As a **student**, I want to submit a document request or a library hold and check its state from LINE, Telegram or the website at any hour, so that I do not travel to campus only to ask whether it is ready.
4. As a **staff member**, I want to take over a conversation and see everything the student already explained, so that a student whose case I inherit does not have to start again.
5. As a **staff member**, I want to ask "show me today's pending requests" and mark one ready without leaving the chat, so that routine administration costs me no context switch.

5.2 Detailed Use Case

UC-01: Reserve a Bookable Resource

ACTORS

Primary: Student. Secondary: the resource owner, who is an advisor, a facility manager or the library. Supporting: Keycloak, PostgreSQL, the Ollama model host.

PRECONDITIONS

The student holds an administrator-provisioned account and is signed in through Keycloak. The resource owner has published at least one availability window. The application, the database and the model host are reachable.

POSTCONDITIONS

Success: exactly one reservation row exists for that slot, owned by this student, the slot is no longer available, and an audit record and a confirmation message exist. Failure: no reservation row is created and the slot keeps its previous state.

MAIN FLOW

1. The student writes in the chat that they want a study room on Thursday afternoon.
2. The student supervisor classifies the message as a booking intent and routes it to the Booking agent.
3. The Booking agent reads the student identifier from the verified access token.
4. The agent identifies the resource type and queries free slots for the requested date range.
5. The agent presents up to five options with resource name, day and time.
6. The student selects one option.
7. The agent opens a transaction, locks the slot row, confirms it is still free, writes the reservation and commits.
8. The agent writes an audit record of actor, action, target and timestamp.
9. The system confirms the resource, day, time and location, and states the cancellation cut-off.

ALTERNATIVE AND EXCEPTION FLOWS

- **A1. Slot taken during selection (step 7).** The lock shows the slot is no longer free. The transaction rolls back, the agent reports the conflict, re-queries availability and returns to step 5. No reservation is written.
- **A2. No availability in the range (step 4).** The agent reports that no slot is free, offers the next available period and returns to step 4 with the new range.
- **A3. Ambiguous resource or date (step 4).** For "book a room" the agent asks which resource type is meant. For "next week" near a month boundary the agent states the exact date range it will use and asks for confirmation before querying.
- **E1. Session expired (step 3).** The token is invalid. The system does not guess an identity. It asks the student to sign in again and discards the pending action.
- **E2. Overlapping reservation already held (step 7).** The agent refuses the second reservation, shows the existing one and offers to reschedule it.
- **E3. Model host unreachable, or malformed tool arguments (any step).** The adapter retries twice and then posts a plain-text message within 10 seconds offering a human handoff. For malformed arguments the validation layer rejects the call and retries once, then escalates to a human rather than guessing.
- **E4. Database unavailable (step 7).** The transaction rolls back. The system reports that the booking did not complete and instructs the student to try again. No partial state is written.

Section 6: Controlled Scope Statement

6.1 In-Scope: Project I Deliverables

1. A public university website with program, admission, announcement and contact pages, plus an authenticated student portal and a staff console, delivered as one React application with Keycloak role-gated routes.
2. Authentication and role-based access through Keycloak for student, staff, advisor and administrator, with accounts provisioned by the administrator.
3. A student supervisor routing to five agents: **Knowledge** (answers from the English and Thai corpus with a citation), **Records** (read-only personal data), **Booking** (reserve, reschedule and cancel any bookable resource with transactional slot locking), **Request** (submit and track any service request through four states), and **Catalog** (library search and availability).

4. A staff supervisor with a Queue agent and an Action agent, requiring explicit confirmation before any action affecting more than one record.
5. Human handoff through Chatwoot, where the agent stops replying while a staff member is handling the conversation, across three channels sharing one conversation history: web widget, LINE and Telegram.
6. Running the model locally through Ollama behind one adapter file, with the model chosen by testing in Week 2.
7. An evaluation harness of 50 test questions in English and Thai, 100 labelled routing messages and the access-control negative test suite, plus a Docker Compose deployment that starts the full stack with one command.

6.2 Out-of-Scope: Deferred to Project II or Backlog

1. Payment processing and payment slip reading. The system records a payment reference only, and a human confirms it.
2. Integration with a real student information system, grade entry, result publication, and automated admission or scholarship decisions.
3. Course add and drop execution, and prerequisite or timetable-clash checking, which would need a scheduling engine the project does not build.
4. Full-text retrieval inside library holdings, which would need licensed content. The Catalog agent works on metadata only.
5. Any channel beyond web, LINE and Telegram. Native mobile applications. Voice input. Languages beyond English and Thai.
6. Cloud deployment with a public domain, managed TLS and a paid inference provider. Model training or fine-tuning, learning-management functions, analytics dashboards and multi-institution scaling.

6.3 Constraints, Assumptions and Limitations

Constraints. Ten working weeks for a four-person team, with W10 holding no new functionality. Zero budget, which excludes any paid model API, managed database or paid hosting. Four laptops, two with 16 GB and two with 8 GB, and no graphics processor, which caps both the model size and the number of services per machine. Docker Compose on a single host with the model on a second host on the local network, so there is no orchestration platform and no horizontal scaling.

Assumptions. An open-weight model that fits in 16 GB of memory is reliable enough at tool calling for this design. We check this with the model test in Week 2 instead of assuming it. Documents we write from public university handbooks are close enough to a real university's documents in length, structure and vagueness. Students use a chat interface without training. Resource owners will publish availability digitally. The team can reach the model host over a shared network or a free mesh network service. The interfaces frozen in W1 remain stable.

Limitations. The prototype runs on team hardware, so it is not continuously available and the demonstration depends on two specific machines. Response times reflect a local model on consumer hardware and will be slower than a hosted service. All records are made up, so we show that the system works, but not how it copes with messy real data such as duplicate or incomplete records. Thai quality is expected to be lower than English quality, and the report states the measured gap rather than hiding it. We test usability with five users, which is enough to find the big problems but is too small a group to be representative. We write the test set ourselves, so two of us score the answers separately. That reduces our own bias but does not remove it.